

SymPy Bug Report

1 Bug 51: False solution for $\text{atan2}(x, 1) = \pi$ in solveset

1.1 Minimal Reproducer

```
from sympy import Eq, S, atan2, pi, solveset, symbols

x = symbols("x")
eq = Eq(atan2(x, 1), pi)
sol = solveset(eq, x, S.Reals)

print("solveset(Eq(atan2(x, 1), pi), x, S.Reals) =", sol)
print("residual at returned value 0 =", (eq.lhs - eq.rhs).subs(x, 0))
print("atan2(0, 1) =", atan2(0, 1))
```

1.2 Observed Behavior

```
SymPy version: 1.14.0
solveset(Eq(atan2(x, 1), pi), x, S.Reals) = {0}
residual at returned value 0 = -pi
atan2(0, 1) = 0
```

SymPy reports $x = 0$ as a real solution. Direct substitution shows that this point does not satisfy the equation: $\text{atan2}(0, 1) = 0$, so the left-hand side minus the right-hand side is $-\pi$.

1.3 Expected Behavior

The correct solution set is $\emptyset$. For every real x , the value $\text{atan2}(x, 1)$ is the principal argument of the point $(1, x)$, which lies in the right half-plane. Its value is therefore strictly between $-\pi/2$ and $\pi/2$, and it can never equal π .

1.4 Mathematical Explanation

Because the second argument is positive,

$$\text{atan2}(x, 1) = \arctan(x)$$

for real x , using the principal real branch of $\arctan$. This branch has range

$$-\frac{\pi}{2} < \arctan(x) < \frac{\pi}{2}.$$

The equation $\text{atan2}(x, 1) = \pi$ is therefore equivalent to $\arctan(x) = \pi$, which has no real solution because π is outside the range of $\arctan$.

The returned set $\{0\}$ is not a harmless branch representation or an alternate form of the empty set. It is a false positive: substituting $x = 0$ gives $\text{atan2}(0, 1) = 0$, not π .

1.5 Source-Level Diagnosis

The source-level diagnosis identifies the relevant solver logic in `sympy/solvers/solveset.py`, especially the generic inverse-function branch in `_invert_real` near lines 230–238 and the final checking logic near lines 1395–1399. The traced call path is as follows. For a solve over $S.\text{Reals}$, `solveset` first replaces the user symbol by a real dummy in `sympy/solvers/solveset.py`. Then $\text{atan2}(_R, 1)$ evaluates to $\arctan(_R)$ in `sympy/functions/elementary/trigonometric.py`, because the second argument is positive. The solver then handles $\arctan(_R) - \pi = 0$.

The problematic rule is the generic inversion step in `_invert_real`:

```
imageset(Lambda(n, f.inverse()(n)), g_ys)
```

For $f = \arctan$, this applies $f^{-1} = \tan$ to the right-hand side and maps π to $\tan(\pi) = 0$. This inversion is not reversible unless the right-hand side is restricted to the actual real range of $\arctan$, namely $(-\pi/2, \pi/2)$. The diagnosis also notes that the later `_check` path only verifies definedness and finiteness through `domain_check`; it does not verify that the equation residual is zero. As a result, the spurious candidate 0 survives.

A plausible fix direction supported by the diagnosis is to intersect the right-hand-side set with the range of the inverse trigonometric function before using its `.inverse()` method. For $\arctan$, values outside $(-\pi/2, \pi/2)$ should be rejected before applying $\tan$. A stronger finite-solution residual check would also catch this class of false positives.

1.6 Additional Failing Instances

The checked cases cover the representative case and five additional instances of the same pattern, $\text{atan2}(ax, b) = \pm\pi$, with positive b . In each case, $b > 0$ places the point in the right half-plane, so the principal value is again in $(-\pi/2, \pi/2)$. The equations with right-hand side π or $-\pi$ should all have empty real solution set.

Input / Expression	SymPy behavior	Expected behavior
<code>solveset(atan2(x, 1) = π, x, $\mathbb{R}$)</code>	returns $\{0\}$; residual at 0 is $-\pi$	$\emptyset$
<code>solveset(atan2(x, 2) = π, x, $\mathbb{R}$)</code>	returns $\{0\}$; residual at 0 is $-\pi$	$\emptyset$
<code>solveset(atan2(2x, 1) = π, x, $\mathbb{R}$)</code>	returns $\{0\}$; residual at 0 is $-\pi$	$\emptyset$
<code>solveset(atan2(x/3, 1) = π, x, $\mathbb{R}$)</code>	returns $\{0\}$; residual at 0 is $-\pi$	$\emptyset$
<code>solveset(atan2(-x, 1) = π, x, $\mathbb{R}$)</code>	returns $\{0\}$; residual at 0 is $-\pi$	$\emptyset$
<code>solveset(atan2(x, 1) = $-\pi$, x, $\mathbb{R}$)</code>	returns $\{0\}$; residual at 0 is π	$\emptyset$

1.7 Related Issues Assessment

The best-effort deduplication check found public issues in the same broad family: SymPy issue #6495 discusses `atan2` simplification and inverse-trigonometric branch ambiguity, pull request #21297 concerns improved trigonometric solving in `solveset`, and issue #12527 reports a different `solveset` trigonometric domain-handling problem. None of the reported matches identifies `solveset(Eq(atan2(x, 1), pi), x, S.Reals)` returning `{0}`, or the specific false positive caused by reducing `atan2(x, 1)` to `arctan(x)` and applying `tan` to an out-of-range right-hand side. The deduplication verdict is therefore a new instance of a known failure mode: the general branch/range issue is known, but this exact reproducible case remains previously unreported and individually actionable as an issue and regression test.

1.8 Proposed SymPy Regression Test

```
import pytest
from sympy import Eq, Rational, S, atan2, pi, solveset, symbols

x = symbols("x")

@pytest.mark.parametrize(
    "scale,denom,sign",
    [
        (S.One, S.One, S.One),
        (S.One, S(2), S.One),
        (S(2), S.One, S.One),
        (Rational(1, 3), S.One, S.One),
        (-S.One, S.One, S.One),
        (S.One, S.One, -S.One),
    ],
)
def test_solveset_atan2_rejects_values_outside_principal_range(scale, denom, sign):
    assert solveset(Eq(atan2(scale*x, denom), sign*pi), x, S.Reals) == S.EmptySet
```